

Soll-Konzept (Zielkonzept)

1. Zielbild und Abgrenzung

1.1 Zweck und Zielsetzung

Zweck des Projekts ist die Entwicklung einer terminalbasierten CLI-Anwendung zur automatisierten Erstellung containerisierter Entwicklungsumgebungen.

Problemstellung:

Die initiale Einrichtung moderner Softwareprojekte ist häufig zeitaufwendig und fehleranfällig. Entwickler müssen verschiedene Technologien (Frontend, Backend, Datenbank) manuell auswählen, konfigurieren und miteinander integrieren. Zusätzlich erfordert die Containerisierung (z. B. mit Podman) technisches Fachwissen und erhöht die Komplexität der Einrichtung. Dies führt zu ineffizienten Entwicklungsstarts und inkonsistenten Projektstrukturen.

Zielsetzung:

- Fachliche Ziele:
 - Vereinfachung und Beschleunigung des Projekt-Setups
 - Reduzierung von manuellen Konfigurationsfehlern
 - Bereitstellung einer einheitlichen und standardisierten Projektstruktur
 - Unterstützung gängiger Technologie-Stacks für typische Webanwendungen
- Technische Ziele:
 - Entwicklung einer stabilen und benutzerfreundlichen CLI-Anwendung
 - Automatisierte Generierung von Projektstrukturen und Quellcode
 - Integration einer automatischen Containerisierung mittels Podman
 - Erstellung einer lauffähigen Entwicklungsumgebung über „podman-compose up“
 - Sicherstellung der grundlegenden Kommunikation zwischen Frontend, Backend und Datenbank
 - Modularer Aufbau zur einfachen Erweiterung um weitere Technologien

1.2 Zielgruppe und Stakeholder

Primäre Nutzer:innen:

- Softwareentwickler:innen, die schnell eine standardisierte Entwicklungsumgebung aufsetzen möchten
- Studierende und Auszubildende im IT-Bereich, die Projektstrukturen und moderne Technologien erlernen oder prototypisch einsetzen möchten
- Entwickler:innen, der reproduzierbare Setups für Projekte oder Tests benötigen

Weitere Stakeholder:

- Prüfer:innen (z. B. im Rahmen einer Projektarbeit oder Abschlussprüfung), die die fachliche und technische Umsetzung bewerten
- Auftraggeber:in bzw Projektbetreuer, die die Anforderungen definiert und die Abnahme durchführt und die das Projekt begleiten und fachliches Feedback geben

1.3 Projektumfang

Im Projekt enthalten (In-Scope):

Die Entwicklung einer terminalbasierten CLI-Anwendung zur automatisierten Erstellung einer containerisierten Entwicklungsumgebung. Dies umfasst:

- Interaktive Konfiguration von Frontend, Backend und Datenbank (mind. 2 Frontend-, 2 Backend-Optionen, 1 SQL und 1 NoSQL Datenbank)
- Automatische Generierung einer standardisierten Projektstruktur
- Initialisierung der Projekte inklusive Installation von Abhängigkeiten
- Erstellung von Containerfiles und einer podman-compose.yml
- Startfähigkeit der Umgebung über „podman-compose up“
- Sicherstellung der grundlegenden Erreichbarkeit der Services
- Umsetzung als funktionaler Prototyp mit modularer Erweiterbarkeit

Nicht Bestandteil (Out-of-Scope):

- Produktionsreife Systeme oder vollständige Deployment-Pipelines
- Grafische Benutzeroberflächen (nur CLI)
- Cloud- oder Hosting-Integration
- Erweiterte Sicherheitsmechanismen
- Persistente Benutzerverwaltung mit Speicherung von Individueller Konfigurationen

1.4 Annahmen und Rahmenbedingungen

Technische Rahmenbedingungen:

- Die Anwendung wird als CLI-Tool entwickelt und über das Terminal ausgeführt
- Zielbetriebssystem ist primär MacOS da die Tests auf anderen Betriebssystemen den zeitlichen Projektrahmen überschreiten würden
- Podman ist auf dem Zielsystem installiert und funktionsfähig
- podman-compose ist verfügbar und korrekt eingerichtet
- Je nach Implementierung ist eine Laufzeitumgebung erforderlich (z. B. Node.js oder Python)
- Internetverbindung ist für die Installation von Abhängigkeiten notwendig
- Es werden ausschließlich frei verfügbare und lizenzfreie Softwarekomponenten verwendet

Zeitliche und organisatorische Rahmenbedingungen:

- Die interne Deadline des Projekts ist der einschließlich der 07.04.2026
- Die Gesamtbearbeitungszeit beträgt 80 Stunden
- Es erfolgt keine produktive Nutzung, sondern die Entwicklung eines funktionalen Prototyps
- Abstimmungen erfolgen mit den Projektbetreuern die gleichzeitig auch die Auftraggeber sind

2. Fachliches Soll-Konzept

2.1 Use Cases

UC1: Erstellung einer Entwicklungsumgebung

Akteur: Auszubildende:r / Studierende:r

Beschreibung:

Der Auszubildende:r / Studierende:r ist anfänger im Webentwicklungs Bereich. Um erste Erfahrungen in der Entwicklung eines Systems welches mehr als die Visuelle gestaltung eines Frontends umfasst wird die lernende Person beauftragt eine simple CRUD App mit Frontend und Backend zu erstellen.

Ablauf:

1. CLI wird gestartet
2. Lernende person wählt Frontend, Backend und Datenbank
3. Konfiguration wird bestätigt
4. Projekt wird generiert

Ergebnis:

Eine vollständige, lauffähige Projektstruktur wird erstellt und die lernende Person kann sich auf das Erstellen einer CRUD-App fokussieren.

UC2: Isolierte Entwicklung und Test einer Funktion

Akteur: Erfahrene:r Entwickler:in

Beschreibung:

Ein:e erfahrene:r Entwickler:in möchte eine neue Funktion oder ein Konzept unabhängig von einem bestehenden Großprojekt entwickeln und testen. Ziel ist es, Abhängigkeiten und Komplexität zu vermeiden, um sich ausschließlich auf die Logik und das Verhalten der Funktion zu konzentrieren.

Ablauf:

1. CLI wird gestartet
2. Entwickler:in wählt gezielt einen minimalen Technologie-Stack
3. Konfiguration wird bestätigt
4. Projekt wird generiert

Ergebnis:

Eine schlanke, containerisierte Entwicklungsumgebung steht zur Verfügung, in der die gewünschte Funktion unabhängig von bestehenden Systemen entwickelt und getestet werden kann.

2.2 Nutzerfluss (Happy Path)

1. Die CLI-Anwendung wird über das Terminal gestartet
2. Der Benutzer wird durch einen interaktiven Konfigurationsassistenten geführt
3. Auswahl von Frontend, Backend und Datenbank erfolgt
4. Die Konfiguration wird validiert
5. Die Projektstruktur wird automatisch generiert
6. Abhängigkeiten werden installiert
7. Containerfiles und podman-compose.yml werden erstellt
8. Der Benutzer startet die Umgebung mit „podman-compose up“
9. Alle Services laufen und sind erreichbar

2.3 Eingaben

- Eingabe des Projektnamens
- Schrittweise Auswahl der Technologie für Frontend, Backend, Datenbank

2.4 Ausgaben und Artefakte

Generierte Ordnerstruktur:

- /frontend
- /backend
- /database
- Konfigurationsdateien im Projektverzeichnis

Konfigurationsdateien:

- Projektbezogene Konfigurationsdatei
- podman-compose.yml

Containerisierung:

- Containerfiles für Frontend und Backend
- Service-Definitionen für alle Komponenten

Konsolenoutput:

- (Fortschrittsanzeigen während der Generierung)
- Zusammenfassung der gewählten Konfiguration
- Hinweise zu Start und Nutzung
- Fehlermeldungen bei Problemen

3. Technisches Soll-Konzept (Wie wird es gebaut?)

3.1 Architekturübersicht

Die Anwendung ist modular aufgebaut und besteht aus mehreren klar getrennten Komponenten:

- CLI-Interface: Einstiegspunkt und Benutzerinteraktion
- Konfigurationsmodul: Verarbeitung und Validierung der Eingaben
- Generator: Erstellung der Projektstruktur und Dateien
- Containerisierungsmodul: Erstellung der Containerkonfiguration

Die Module kommunizieren über definierte Schnittstellen und tauschen strukturierte Konfigurationsdaten aus.

3.2 Modulstruktur und Verantwortlichkeiten

CLI-Entry / Command Root:

- Startpunkt der Anwendung
- Verarbeitung von Argumenten und Anzeige von Hilfe

Konfigurationsassistent:

- Interaktive Benutzerführung
- Validierung der Eingaben

Projektgenerator:

- Erstellung der Ordnerstruktur
- Einfügen von Templates
- Ausführen von Setup-Befehlen

Containerisierung:

- Erstellung von Containerfiles
- Generierung der podman-compose.yml

3.3 Datenmodell / Konfigurationsschema

Beispielhafte Struktur:

```
{  
  
  "projectName": "string",  
  
  "frontend": "string",  
  
  "backend": "string",  
  
  "database": "string",  
  
  "portConfig": {  
  
    "frontend": "number",  
  
    "backend": "number"  
  }  
}
```

Speicherort:

- Im Projektverzeichnis als Konfigurationsdatei

3.4 Schnittstellen zwischen Modulen

- CLI → Konfigurationsmodul:

Übergabe von Eingaben und Parametern

- Konfigurationsmodul → Generator:

Übergabe eines validierten Konfigurationsobjekts

- Generator → Containerisierung:

Übergabe der Projektstruktur und Service-Definitionen

3.5 Erweiterbarkeit (Plug-in/Interface-Konzept)

- Neue Technologien werden über definierte Module integriert
- Jede Technologie besitzt:
 - Templates
 - Konfigurationsdefinition
 - Setup-Skripte

Konventionen:

- Einheitliche Ordnerstruktur
- Standardisierte Schnittstellen für neue Module
- Klare Namenskonventionen

3.6 Logging und Fehlerhandling

Log-Level:

- INFO: Standardinformationen
- WARN: Hinweise auf mögliche Probleme
- ERROR: Fehler mit Abbruch oder Einschränkungen

Fehlermeldungen:

- Klar verständlich
- Mit konkreten Lösungsvorschlägen

4. Qualität, Tests und Abnahme

4.1 Qualitätsanforderungen (nicht-funktional)

Wartbarkeit:

- Modularer und strukturierter Code
- Klare Trennung von Logik und Konfiguration

Portabilität:

- Ausführung auf unterstützten Systemen möglich

Benutzerfreundlichkeit:

- Intuitive CLI-Bedienung
- Verständliche Eingaben und Ausgaben

4.2 Teststrategie

Unit Tests:

- Für zentrale Funktionen (z. B. Validierung, Generatorlogik)

Integrationstests:

- Prüfung der Projektgenerierung
- Validierung der podman-compose.yml

Manuelle Tests:

- Durchspielen typischer Nutzerabläufe
- Checkliste zur Abnahme

4.3 Abnahmekriterien

- Wie im Pflichtenheft (Abschnitt 7.3)

5. Offene Punkte und Entscheidungen

5.1 Offene Fragen

- Welche Programmiersprache wird final verwendet (Node.js oder Python)?
- Soll eine Konfigurationsdatei verpflichtend oder optional sein?
- Welche konkreten Technologien werden im Prototyp unterstützt?

5.2 Entscheidungen (mit Begründung)

Entscheidung:

Verwendung von Podman anstelle von Docker

Begründung:

- Betriebliche Anforderung da keine Docker Lizenzen zur Verfügung stehen
- Fokus auf containerbasierte Entwicklung ohne Docker-Abhängigkeit
- Moderne und daemonlose Architektur

Alternativen:

- Docker durch Lizenzgebühren nicht für das Projekt geeignet

Anhang